Fuzix for the Pico Computer

Unix and BBC BASIC on the Pico Computer 2 and 3

Release v0.6 — August 2026

Contents

1 Introduction

Fuzix is Alan Cox's compact V7-style Unix. This port makes it a third first-class environment for the Pico Computer, beside MMBasic and MicroPython: a genuine multi-tasking Unix booting from SD card, with a full set of classic utilities — and R. T. Russell's BBC BASIC as its flagship application, complete with BBC graphics modes on the HDMI display, the four-channel BBC sound system on the audio DAC, joystick and analogue inputs, and a spare serial port.

It also compiles on its own. `cc` is a complete C89 toolchain running on the hardware — not a cross-compiler — and it generates native ARM code, so the machine builds its own programs, at its own speed, with no PC involved. `mmbc` puts MMBasic in front of it: an MMBasic program translates to C, compiles, and runs as a native program several times faster than the interpreter it was written for.

One kernel image serves both machines. At boot it probes GP27 for the DS3231's 32 kHz clock (the same test MMBasic and MicroPython use): present means Pico Computer 3, absent means Pico Computer 2. The practical difference is the SD card wiring, handled automatically; the boot banner names the detected board.

Headline specification as configured here:

- CPU at 375 MHz (the XGA-capable clock, as MMBasic uses)
- 312 KB of program RAM managed as 4 KB pages, with the 8 MB PSRAM behind it — up to 30 concurrent processes, and a single process may use all of program RAM. A swapped-out process is a PSRAM allocation the size of the process, not a slot on a device, so nothing is reserved for one that does not exist
- Programs get their heap from the PSRAM too, so a BASIC array or a C `malloc` is limited by the 8 MB rather than by the process
- Root filesystem on SD card; the on-board NAND flash holds a recovery system
- 80×40 colour ANSI console on HDMI, mirrored to the USB-C serial port, with USB keyboard support (six layouts)
- Pre-emptive multitasking: a runaway program can always be stopped from the keyboard
- A self-hosted C89 compiler generating native ARM code, and an MMBasic translator in front of it — both run on the machine itself
- MMBasic's own full-screen editor, `mmedit`, so BASIC is written, translated, compiled and run without leaving the machine

1.1 New in v0.6

This release is about memory. A translated BASIC program could not hold a framebuffer-sized array, and a large one could not be loaded at all; both are fixed, and the same work made everything faster.

- **BASIC arrays and strings live in the PSRAM heap.** They used to sit in the 48 KB the interpreter gave a program, so a 38,400 byte array — one MODE 1 framebuffer — did not fit at all. Globals go in one block taken once; `LOCAL` arrays and strings get a block per invocation, freed on the way out, which is what makes recursion correct. Simple variables stay where they are: they are the hot ones, and SRAM is 3.7× faster to reach than PSRAM.
- **Programs are 15% faster.** A program address is now a machine address rather than an offset into a 48 KB window, so every access stops paying for the conversion. The solar eclipse went from 3.31 to 2.80 seconds, and the heap change above costs about half a percent on top.
- **A process may use all of program RAM.** The old 256 KB ceiling was the size of a fixed swap slot, and swap has not worked that way for a while.

- **`SYSTEM`, `SAVE IMAGE` and `LOAD IMAGE` work from large programs again.** `fork` needed the process resident twice over, so anything past half of memory could not do it. The parent is now staged into PSRAM instead.
- **A memory leak fixed** that cost 132 KB — of 312 KB — every time a program failed to start, for the rest of that boot.
- **A C library bug fixed** that made `fread` and `fseek` disagree about a file's position and quietly return the wrong bytes. It needed a file small enough to fit one buffer, which is why it went unnoticed: it took down the compiler on a 305-byte object while 100 KB ones were fine. It is not specific to this machine and has been reported upstream.

1.2 New in v0.5

- **MMBasic draws.** `MODE`, `COLOUR`, `PIXEL`, `LINE`, `CIRCLE` and `RGB()` are translated and run on the HDMI display — a whole shape crosses into the kernel in one call, so a 640-point line costs one syscall rather than 640.
- **`SAVE IMAGE` and `LOAD IMAGE`**, with MMBasic's BMP decoder: 1/4/8/16/24/32-bit files, RLE4 and RLE8, and its dithering modes.
- **`SYSTEM`** — a BASIC program can run another program and wait for it, which is how `SAVE IMAGE` and `LOAD IMAGE` are built.
- **`mmedit`**, MMBasic's editor, ported.
- **`TIMER` is a float** off a 64-bit microsecond clock, and **`RND` returns 53 bits** from splitmix64 rather than 15 from the C library.
- The SD root filesystem is now **built from source** by `mksdimage.sh` — it had no build recipe before — and is **64000 blocks** rather than the 65535 that sat on `blkno_t`'s ceiling.
- A **filesystem corruption fixed** that had been destroying cards under repeated large-file rewrites: `f_trunc` freed a file's double indirect block and left the inode still pointing at it. It bit any file over 273 blocks. See the release notes.
- **Panic messages now reach the serial port.** They were being truncated to about eleven characters by an interrupt-driven UART that stopped with the machine.

2 Installing Fuzix

2.1 Flashing the kernel

The kernel is a standard UF2 file, `fuzix.uf2`, flashed exactly like any other Pico Computer firmware:

1. Set the USB HUB switch to **DISABLE** and connect the **Prog** port to your PC.
2. Hold **BOOT**, click **RESET**, release BOOT. A USB drive appears.
3. Drag `fuzix.uf2` onto the drive. The board reboots when the copy completes.
4. Set the HUB switch back to **ENABLE**.

Flashing does not touch the SD card.

2.2 Writing the SD card

Fuzix boots from a ready-made card image, `pc3-sd.img` (about 101 MB). Write it to a card of 128 MB or larger with any raw-image tool — Raspberry Pi Imager ("use custom image"), Win32DiskImager, balenaEtcher, or `dd` on Linux/macOS. **The whole card is overwritten.**

The image lays the card out as three partitions:

Partition	Size	Type	Purpose
1	64 MB	FAT	File interchange with Windows/macOS/Linux
2	32 MB	Fuzix root	The Unix filesystem (boot device `hdb2`)
3	4 MB	0x7F	Reserved

The root filesystem is 64000 blocks of 512 bytes. That is a little short of the 65536-sector partition on purpose: block numbers are 16-bit, so 65535 is the ceiling, and the "no such block" marker is 65535 as well — leaving room below both keeps a stray value recognisable as wrong rather than pointing at a real sector. From v0.5 the filesystem is built from source by `mksdimage.sh`, so the card can be reproduced rather than merely copied.

Partition 1 ships unformatted: format it as **FAT or FAT32 (not exFAT)** in Windows before first use — see section 6. Partition 2 is Fuzix's own filesystem format; no desktop OS can mount it, which is why the FAT partition and the `fat` command exist.

2.3 First boot

Connect a monitor to the HDMI port and/or a terminal program (for example TeraTerm at 115200) to the USB-C console port. Switch on: the kernel banner, board identification and device probe appear on both, then:

```
login: root
```

There is no password. The system arrives at a Bourne shell; the console is an 80×40 colour terminal (termcap type `pc3`) and the USB keyboard and serial input feed the same session interchangeably. Set the keyboard layout in `/etc/rc` (`picoctl keymap uk` by default).

To power off, type `shutdown` (or at minimum `sync`, then wait a moment). After an unclean power-off the next boot repairs the filesystem automatically (`fsck -a -y`).

2.4 Setting the clock

The board's DS3231 battery-backed clock supplies the date and time: at every boot `/etc/rc` runs `setdate`, which reads the chip and sets system time silently. If the chip cannot supply a valid time (fresh battery, or the battery was removed) the same command falls back to prompting on the console:

```
Current date is Mon 2026-07-27
Enter new date:
```

Press Enter to keep a value, or type a new one (`2026-07-28`, then `16:30:00`). To set the clock at any other time run `setdate -u`, which always prompts.

Setting the system time does not touch the chip. To make the time permanent, write it back:

```
# setdate -w
writing
```

That also restarts a stopped oscillator, so the sequence for a new battery is: `setdate -u` (enter the time), then `setdate -w`. The kernel message `oscillator was stopped, time needs setting` at boot means exactly that sequence is wanted.

While running, system time is kept by the crystal-driven timer tick and gently re-synchronised from the DS3231 about once an hour, the same policy as MMBasic. `date` prints the current time.

If a transaction to the chip is ever interrupted at the wrong moment (a reset mid-read), the battery-backed DS3231 can be left jamming the I2C bus - even across power cycles. The kernel detects this at boot (`ds3231: SDA held low, clocking bus free`) and clears it automatically.

2.5 Command history and line editing

At any cooked-mode prompt - the shell, login, most line-oriented programs - the console provides in-line editing and command history:

Key	Action
Up / Down	walk back / forward through previous commands
Left / Right	move within the line
Home / End (or Ctrl-A / Ctrl-E)	start / end of line
Backspace / Delete	delete left / at the cursor
Ctrl-U	discard the line

The history (about 500 commands) lives in a small region reserved at the top of the PSRAM, so it costs no program memory; like the RAM disc it survives a reset but not a power cycle. Programs that take the terminal raw (BBC BASIC, the editor) see every keystroke themselves as usual - BBC BASIC has its own line editor and *EDIT.

2.6 Escape routes

- **Esc** — inside BBC BASIC, stops the running program.
- **Ctrl-** — kernel emergency break: kills the foreground program even if it has the terminal in raw mode, and restores a sane terminal. Use it instead of the RESET button when something wedges.
- `kill` from the shell works on anything; Fuzix pre-emption guarantees a spinning process can't lock you out.

3 The consoles

Everything is mirrored: kernel and program output appear on the HDMI display and the serial console simultaneously, and input is merged from the USB keyboard and the serial line. TeraTerm is resized to 80×40 automatically at boot.

While BBC BASIC has a graphics MODE on screen, the HDMI display belongs to the graphics; text output continues to the serial side as a plain stream, so a transcript survives even a full-screen game.

4 BBC BASIC

Start it by name; leave with `QUIT`:

```
# bbcbasic
BBC BASIC for Fuzix Console v0.50
(C) Copyright R. T. Russell, 2025
>
```

This is R. T. Russell's BBC BASIC (the BBCSDL console edition), so the language is the full modern dialect: 64-bit integers, IEEE double floats, long variable names, `WHILE/ENDWHILE`, multi-line `IF/ELSE/ENDIF`, structures, and the inline assembler. Keywords must be typed in **capitals** (`PRINT`, not `print`); `*LOWERCASE ON` relaxes this. Star commands are case-insensitive.

4.1 Loading programs, including plain text

`LOAD` and `CHAIN` accept **both** program formats:

- the tokenised internal format written by `SAVE` (fast, exact), and
- **plain ASCII listings**, detected automatically. An unnumbered modern-style listing is given line numbers 10, 20, 30…; a listing with its own numbers keeps them. Windows line endings are fine, and typographic characters that desktop editors introduce (curly quotes, non-breaking spaces, tabs) are silently converted to their ASCII equivalents — an invisible "smart" character can otherwise produce a baffling `Syntax error` on a line that looks perfect.

The comfortable round trip: edit `myprog.bas` on your PC, copy it to the FAT partition, then:

```
# fat get myprog.bas
# bbcbasic
>LOAD "myprog.bas"
>RUN
>SAVE "MYPROG"          save tokenised for fast loading next time
```

4.2 Graphics

`MODE 0`–`5` switch the display to 1024×768 and provide the classic screens; `MODE` with any higher number (e.g. `MODE 6`) returns to the text console. Errors leave you at the prompt *in* the current mode, exactly like the original machine.

MODE	Resolution	Colours	Presentation on the monitor
0, 3	640×256	2	Full width (5:8 smooth upscale), full height
1, 4	320×256	4	960×768 — every pixel a crisp 3×3 block
2, 5	160×256	16	960×768 — every pixel a crisp 6×3 block

Coordinates are the authentic 1280×1024 graphics units, origin bottom-left, movable with `VDU 29`. Implemented: `MOVE`, `DRAW`, `PLOT` (move/line families 0–15, points 64–71, filled triangles 80–87 — so `PLOT 85` works), `CLG`, `GCOL`, `COLOUR`, `POINT(x,y)`, `VDU 19` palette changes (instant, no redraw), `TAB(x,y)`, and text printed with the built-in 8×8 font (40 or 80 columns × 32 rows, with scrolling). `MODE 1/4` store 4 bits per pixel, so all 16 logical colours are available there via `VDU 19` even though the default palette is the authentic four.

4.3 Sound

The full BBC sound system plays through the PCM5102 DAC and the 3.5 mm jack:

```
SOUND 1,-15,89,20                              A4 (440 Hz) for one second
ENVELOPE 1,1,4,-4,4,10,10,10,126,-2,0,-10,120,100
SOUND 1,1,100,30                               envelope-shaped note
SOUND &101,-15,53,20:SOUND &102,-15,69,20:SOUND &103,-15,81,20
                                               a synchronised C-E-G chord
```

Channels 1–3 are square-wave tones, channel 0 is noise. Each channel queues up to 8 notes; `&1x` in the channel number flushes the queue, `&1xx`–`&3xx` synchronise channels. `ENVELOPE` implements the three-section pitch envelope and ADSR amplitude, stepped at the authentic 100 Hz. Pitch follows the BBC scale: 4 units per semitone, 89 = A4 = 440 Hz. Durations are in 20ths of a second; 255 means "until further notice". `SOUND` blocks BBC-style when a queue is full (Esc still works).

4.4 ADVAL: joystick, analogue inputs, sound queues

Call	Returns
`ADVAL(0)`	Joystick switches on GP34–37 as a mask: bit 0 = GP34, bit 1 = GP35, bit 2 = GP36, bit 3 = GP37; pressed (grounded) = 1
`ADVAL(1)`–`ADVAL(4)`	Analogue readings of GP41–GP44, 0–65520 (12-bit ADC ×16)
`ADVAL(-1)`	Characters waiting in the keyboard buffer
`ADVAL(-5)`–`ADVAL(-8)`	Free queue slots on sound channels 0–3
`ADVAL(-9)`	Hardware microsecond counter (64-bit)

The joystick inputs have pull-ups and Schmitt-trigger inputs enabled; wire switches directly to ground. The analogue inputs are 3.3 V full-scale.

`ADVAL(-9)` is the benchmarking clock — the RP2350's free-running microsecond timer, far finer than `TIME`'s centiseconds (which tick in tenths of a second on this kernel). MMBasic-style usage:

```
DEF FNtimer = ADVAL(-9)
T% = FNtimer
REM ... code under test ...
PRINT (FNtimer - T%) / 1000; " ms"
```

Each reading costs one system call (a few tens of microseconds) - negligible over millisecond-scale measurements. The value is the full 64-bit hardware counter (BBC BASIC integers are 64-bit), counting microseconds since power-on: it never wraps in practice.

4.5 Serial port

A second serial port lives on the I/O header: **TX = GP0, RX = GP1** (3.3 V logic, no flow control), visible as `/dev/tty2`. BBC BASIC reaches it as a *port channel* — raw and unbuffered:

```
*stty 9600 </dev/tty2
ch% = OPENUP("/dev/tty2")
BPUT#ch%,65                    send a byte
X% = BGET#ch%                  receive a byte (waits for one)
CLOSE#ch%
```

Any `stty` setting applies (baud rate, parity, size). The same `OPENIN`/`OPENUP`/`BGET#`/`BPUT#` mechanism works on any `/dev` device; ordinary file names get the buffered 256-byte file channels instead, exactly as on the original machine.

4.6 Star commands and the shell

The built-in set includes `*DIR`/`*.`, `*CD`, `*TYPE`, `*COPY`, `*DELETE`/`*ERA`, `*MKDIR`, `*RMDIR`, `*KEY`, `*SPOOL`, `*EXEC`, `*LOWERCASE`, `*TEMPO`, `*QUIT`. Anything unrecognised is passed to the Unix shell, so `*ls -l`, `*stty 9600 </dev/tty2` and even `*fat get demo.bas` work from inside BASIC.

4.7 The inline assembler

The assembler between `[` and `]` targets the machine it runs on: **ARM Thumb (v6-M subset)**, not 6502. `CALL` and `USR` work as documented for BBCSDL's ARM editions. Machine code written for a BBC Micro will not run; this is the same situation as every modern BBC BASIC.

4.8 Differences from the original BBC Micro

Better than the original:

- ~110 KB of program workspace (about 80 KB when graphics are in use) against the Model B's 32 KB shared with the screen
- 64-bit integers, double-precision floats, the modern language
- Long file names on a hierarchical filesystem
- The machine multitasks underneath — BASIC is just a process

Not (yet) present, compared with an original machine or full BBCSDL:

- **MODE 7** teletext (MODE 6 and 7 return to the console)
- `VDU 5` text-at-the-graphics-cursor; user-defined characters (`VDU 23`) are accepted but not rendered
- PLOT families beyond points/lines/triangles: no circles, arcs, ellipses, flood fill or rectangle/parallelogram fills yet
- `GCOL` action modes 1–4 (OR/AND/EOR/invert) plot as mode 0 (set)
- Flashing colours (8–15 map to their steady equivalents)
- `*FX` calls, `INKEY` with negative arguments (keyboard scanning), and the 6502 `CALL` interface
- `TIME` advances in 0.1 s steps (the kernel clock granularity)
- Cassette/Econet/ROM filing systems, for obvious reasons

5 The C compiler

The machine compiles C on its own, with no PC involved. `cc` is Alan Cox's Fuzix Compiler Kit, retargeted here to a bytecode machine.

```
# cat > hello.c
#include <stdio.h>

int main(void)
{
        printf("hello from Fuzix\n");
        return 0;
}
^D
# cc hello.c
# ./hello.bc
hello from Fuzix
```

`cc prog.c` writes `prog.bc` and makes it executable, so `./prog.bc` runs it directly — the object starts with `#!/usr/bin/bcrun`, and the kernel does the rest. `bcrun prog.bc` still works and is the way to run an object that has lost its execute bit. Options are `-o name`, `-v` to show each pass as it runs, and `-k` to keep the intermediates. `bcdump prog.bc` disassembles.

`cc` is a driver: it runs `cpp`, then the three compiler passes from `/usr/lib/cc`. The passes cannot be driven from the shell by hand — two of them read back from their own standard output, which needs a redirection the Bourne shell here does not have. Like BBC BASIC, the compiler lives on the SD card root and is not in the NAND recovery system.

A small program takes about a second to compile. The dots `cc` prints are progress, one per top-level declaration.

5.1 What it compiles

C89, plus declarations after statements — the one C99 borrowing, because refusing it is a nuisance out of proportion to the standard it comes from. Everything else is ANSI C as of 1989: the full type system, `struct` and `union` including passing and returning them by value, `enum`, `typedef`, function pointers, `switch`, `goto`, all the usual operators, 64-bit `long long`, and double-precision floating point.

The compiler is checked against gcc as an oracle: 165 of the 175 applicable tests in the public `c-testsuite` conformance suite pass with byte-identical output, and a set of samples in the source tree is diffed against gcc on every change — on the board as well as on the host.

5.2 Headers and the runtime library

`/usr/lib/cc/include` holds `stdio.h`, `stdlib.h`, `string.h`, `math.h`, `assert.h`, `limits.h` and `stddef.h`. These describe what `bcrun` actually provides, and they are deliberately **not** `/usr/include`, which describes the Fuzix C library used by native binaries.

Available: `printf sprintf puts putchar`; `fopen fclose fread fwrite fgetc getc fputc putc fgets fputs fprintf feof fseek ftell rewind fflush remove`; `malloc calloc realloc free exit atoi abs`; `strlen strcpy strncpy strcat strcmp strncmp strchr strrchr memset memcpy memmove memcmp`; and from `math.h`, in double precision, `sin cos tan asin acos atan atan2 sinh cosh tanh sqrt exp`

`log` `log10` `pow` `floor` `ceil` `fabs` `fmod`. Each of those is a native function inside `bcrun`, so it runs at machine speed whatever the calling code is.

`malloc` takes its memory from the PSRAM, not from the program's own 312 KB, so a C program can allocate far more than it could hold itself. It is asked for once when the program loads; set `BCRUN_HEAP` to a size in KB in the environment to change how much.

The raw system calls `open` `creat` `close` `read` `write` `lseek` `unlink` are also linked in, but no header declares them — declare them yourself if you want them. Two extras reach the hardware: `time_us()` returns the microsecond counter, and `adval(n)` is the BASIC `ADVAL` function (joystick, analogue inputs, sound queues).

5.3 Limitations

These are worth knowing before you start a large program.

- **One source file per program.** There is no assembler and no linker: `cc2` writes a loadable object directly, so nothing can be linked together. `cc` rejects a second source file rather than pretending.
- **Bitfields are not implemented.** `unsigned flags : 3;` is refused with a diagnostic. This is the only part of C89 missing.
- **`&` on a library function does not work.** A runtime function is resolved by index and has no address, so `&printf` cannot be represented. `bcrun` refuses such a program by name when it loads it, rather than running it with something wrong in place of an address. Pointers to *your own* functions are entirely fine.
- **No wide strings.** `L'x'` is accepted (and equals `'x'` here); `L"..."` is refused rather than quietly returned as a narrow array.
- **`printf` rounds halves up.** `%.2f` of `0.125` gives `0.13` where a full C library gives `0.12`. A deliberate, documented difference: matching the round-half-to-even rule needs exact decimal expansion.

5.4 Speed

`cc2` translates each function it compiles into native Thumb-2 for the Cortex-M33 and stores that in the object beside the bytecode; `bcrun` runs the native code and keeps the interpreter for whatever did not translate. Nothing needs to be asked for — it is simply what the compiler does.

Dhrystone 2.1, compiled on the machine, runs at about **90,000 Dhrystones/second**, which is a quarter of the same benchmark cross-compiled by `gcc -O2` for the same chip (379,000). Individual loops do better: a sieve, a shell sort and an xorshift generator all land within a factor of two or three of gcc, and double-precision arithmetic uses the RP2350's hardware co-processor through the same routines the rest of the system uses.

A program can be forced to interpret with `BCRUN_BYTECODE=1` in the environment, which is how the native code is checked against the interpreter — the two must agree exactly.

What the compiler is *for* is being able to write and build real programs on the machine itself — utilities, file handling, glue, and now whole applications where the speed matters. Nothing else on the Pico Computer can do it: this is a complete toolchain that runs on the hardware, not a cross-compiler.

The bytecode is also a deliberately language-neutral intermediate form, so front ends for other languages can share the same back end and runtime. `mmbc`, the MMBasic translator described next, is the first.

6 MMBasic: the mmbc translator

mmbc translates an MMBasic program into C. cc compiles that C into native ARM code. The result runs as an ordinary program, several times faster than the same source under the MMBasic interpreter, on the same machine at the same clock.

Nothing about it is a cross-development trick: both programs live on the SD card and run on the Pico Computer.

6.1 The three commands

```
# mmbc prog.bas          -> prog.c
# cc prog.c              -> prog.bc
# ./prog.bc              runs it
```

mmbc writes prog.c next to the source and says so. -o name.c puts it somewhere else. Every line it cannot translate is reported with its line number and left as a comment in the C, so a program that uses something unsupported still produces a translation you can read, and tells you exactly where it gave up.

On the machine, mmbc writes C for the machine's own compiler. On a host it writes the slightly different C that gcc prefers; --fcc and --gcc force either form, which only matters if you are moving the generated C between the two.

6.2 A first program

```
# cat > hello.bas
Print "Hello from MMBasic"
For i = 1 To 5
  Print i, i * i
Next i
^D
# mmbc hello.bas
wrote hello.c
# cc hello.c
....
# ./hello.bc
Hello from MMBasic
 1       1
 2       4
 3       9
 4       16
 5       25
```

The dots from cc are one per top-level declaration — a translated program carries the MMBasic runtime's declarations with it, so there are a couple of hundred of them even for four lines of BASIC. That is also why the first compile takes longer than the size of your program suggests: about half a minute here, and much the same for anything small.

Print with a comma gives MMBasic's tabbed columns, as above.

6.3 A worked example: the KnivD benchmark

This is a published MMBasic benchmark, unmodified. It runs four thirty-second rounds and reports a score.

```
# cat bench.bas
Print "MMBASIC benchmark (C) KnivD 2016"
Dim integer t, i=0
Dim float x(1000), f=0.0
Dim string s=""
Print "Calculating... ";
count%=0
Do
  s=""
  f=0.0
  i=0
  Timer =0
  Do While Timer<30000
    i=i+2 : f=f+2.0002
    If (i Mod 2)=0 Then
      i=i*2 : i=i\2
      f=f*2.0002 : f=f/2.0002
    End If
    i=i-1
    For t=1 To 100
      f=f-1.0001
      If (f-Int(f))>=0.5 Then
        f=Sin(f*Log(i))
        s=Str$(f,6,6)
      End If
      f=(f-Tan(i))*(Rnd(0)/i)
      If Instr(s,LEFT$(Str$(i),2))>0 Then s=s+"0"
    Next
    x(1+(i Mod 1000))=f
  Loop
  Print Chr$(13)+"Performance: "+Str$((i*1024)\286,8,0)+" grains"
  Inc count%
Loop Until count%=4
End

# mmbc bench.bas
wrote bench.c
# cc bench.c
.....................................................................
# ./bench.bc
MMBASIC benchmark (C) KnivD 2016
Calculating... Performance:    28944 grains
Performance:    28944 grains
Performance:    28944 grains
Performance:    28940 grains
```

MMBasic itself scores about 12,000 grains on the same board at the same clock. Note what the program exercises: 64-bit integers, doubles, string building, `Str$`, `Instr`, `Sin`, `Log`, `Tan`, `Rnd`, a thousand-element array and a timer — all of it translated, compiled and running natively.

6.4 A worked example: something numerical

A longer one. `solar_eclipse.bas`, in `/root/cc`, is a 3,200-line astronomical calculation (Bessel elements, lunar and solar series) that reads a date and prints the circumstances of an eclipse. It is a good test because every digit of its output can be checked against other machines.

```
# mmbc solar_eclipse.bas
wrote solar_eclipse.c
# ./solar_eclipse.bc < solar_eclipse.in
...
event duration          2.61100904 hours
Time taken :     2.803  Seconds
```

The same program takes 12.5 seconds under MMBasic and 8.8 seconds under MicroPython on this hardware; every digit printed is identical in all three. Translating it takes a few seconds. Compiling it is a much longer job than the benchmark above — it is a hundred and forty kilobytes of C — so start it when you have the machine to yourself.

6.5 What the translation looks like

The C is meant to be read. Variables keep their names, control flow keeps its shape, and the MMBasic semantics that C does not share — integer division, `Mod` on negative numbers, string handling, the 64-bit integer/double type rules — are handled by a small runtime inside `bcrun` rather than by rewriting your program into something unrecognisable.

```
# mmbc hello.bas
# head -20 hello.c
```

is worth doing once, to see what your program became.

6.6 Strings, arrays and functions

Strings are MMBasic strings: `Dim string s` gives the usual 255 characters, and `Dim string s length 40` the shorter form. Arrays index from `OPTION BASE` as they should, and `Bound()` reports their limits. `Sub` and `Function` work, including arrays passed by reference and modified in place.

Arrays and strings live in the PSRAM heap, so what limits them is the 8 MB rather than the size of a process. A framebuffer-sized array — `Dim Integer fb(4800)`, 38,408 bytes — is unremarkable now; before v0.6 it would not fit at all. Simple variables stay in the program's own memory, where they are quicker to reach, which is the same split MMBasic itself makes and for the same reason.

`LOCAL` arrays and strings get their own block for each call, released when the routine returns, so a recursive routine sees its own copy at every level. `STATIC` locals are unaffected — they are meant to outlive the call, so they stay where they were.

The one thing to remember is that a translated program is a *program*, not an interactive session: there is no editor, no `RUN`, no immediate mode. You edit the `.bas` file, translate, compile and run — which is also why a translated program can be put in `/usr/bin` and used like any other command.

6.7 Errors

`mmbc` reports what it could not translate, by line, and carries on:

```
# mmbc gpio.bas
2 line(s) could not be translated and were commented out:
  line 2: expected '='
  line 3: 'pin' is not an array
wrote gpio.c
```

The reasons are the translator's own — it reads `SetPin GP1, DOUT` as far as it can and then says what it wanted to see — so they name the symptom rather than the statement. The lines themselves appear in the C as comments, with the original text, which is the quickest way to see what was dropped:

```
    /*     SetPin GP1 , DOUT */
```

The translation still happens and everything else works. `--strict` stops on the first such line instead, and `--report` lists them again at the end along with any implied global variables.

Appendix C lists what is covered.

6.8 Graphics

A translated program draws on the HDMI display with MMBasic's own statements:

```
MODE 2
COLOUR RGB(YELLOW)
LINE 0, 0, 319, 239
CIRCLE 160, 120, 60, , , RGB(WHITE), RGB(BLUE)
PIXEL 10, 10, RGB(RED)
```

Two modes are available, chosen to match the VGA builds of MMBasic and the first two HDMI modes:

`MODE`	Resolution	Colours
`MODE 1`	640×480	monochrome
`MODE 2`	320×240	16, from MMBasic's RGB121 set

`RGB()` takes the usual named colours and `RGB(r,g,b)`. `COLOUR` sets the current drawing colour, which every statement uses when no colour is given. `LINE`, `CIRCLE` and the rest take MMBasic's argument order, including the blank arguments — `CIRCLE x, y, r, , , fill` is written exactly as MMBasic writes it.

The drawing primitives live in `mmb_gfx.h` as static functions, so a program that never draws a circle does not carry the circle code: `cc` discards a file-scope static nothing calls. And a whole shape crosses into the kernel in a single call, so a 640-point line is one syscall, not 640 — measured at 71 µs against 433 µs for the naive version.

The geometry is MMBasic's, copied rather than re-derived, down to the dotted appearance of a stretched ellipse. If a picture differs from MMBasic's, that is a bug worth reporting.

One caution. In `MODE 1` the console and the graphics share a single framebuffer, so anything printed — including the cursor — appears on the picture. It is visible in a `SAVE IMAGE` of the whole screen as a difference in the top text line. Restricting the saved region avoids it; directing program output elsewhere is planned.

6.9 Running another program: SYSTEM, SAVE IMAGE, LOAD IMAGE

SYSTEM runs a program and waits for it:

```
SYSTEM "sum", "a.bmp", "b.bmp"
```

Each argument is passed separately, so no shell quoting is involved and a filename with a space in it needs no special care.

SAVE IMAGE and LOAD IMAGE are built on it — they run the saveimage and loadimage programs described in the applications list, which means a BASIC program that never touches an image pays nothing for them:

```
SAVE IMAGE "shot.bmp"                       ' the whole screen
SAVE IMAGE "part.bmp", 160, 120, 320, 240   ' x, y, w, h
LOAD IMAGE "shot.bmp"                       ' at 0,0
LOAD IMAGE "shot.bmp", 160, 120             ' at x,y
```

LOAD IMAGE takes MMBasic's full syntax including the dither mode and the source rectangle:

```
LOAD IMAGE fname$ [,x] [,y] [,mode] [,ximage] [,yimage] [,wimage] [,himage]
```

The decoder is MMBasic's, so it reads 1, 4, 8, 16, 24 and 32-bit BMPs, BI_BITFIELDS, and RLE4/RLE8 compression. Dithering is *optional*, as in MMBasic — omit the mode and the image is mapped without it.

6.10 Practical notes

- One .bas file per program, because cc compiles one file per program. Subs and Functions all live in that file.
- The C file is a normal file — you can keep it, edit it, or hand it to cc on another day.
- mmbc and cc each need a little room. On a machine with other things running, a large program is happier if you are not also holding a big file open in an editor.
- Programs built this way are the same objects cc produces from hand written C, so bcdump disassembles them and BCRUN_BYTECODE=1 forces interpretation for comparison.

7 `mmedit`: MMBasic's editor

MMBasic's full-screen editor is ported and runs as an ordinary Fuzix program, so BASIC is written, translated, compiled and run without leaving the machine:

```
# mmedit prog.bas
```

It is the editor from the firmware, with the same keys:

Key	
`ESC`	leave the editor
`F1`	save
`F2`	save and exit (so a wrapper can run the program)
`F3` / `F6`	find / find again
`F4`	mark — then the cursor keys extend the selection
`F5`	paste
`F7` / `F8`	replace / replace again
`F9` / `F10`	read a file in / write the selection out
`F12` or `Ctrl-A`	beautify: re-indent and case the keywords

In mark mode the legend changes: `DEL` deletes, `F4` cuts, `F5` copies, `F10` exports the selection to a file. The status line shows line, column and INS/OVR, and the function key legend shortens itself to fit the terminal width.

The editor works over the serial port as well as on the HDMI console — it drives a VT100, and the console emits VT100 function key sequences. Files with CRLF line endings are accepted; the CRs are stripped on load, which matters because most files arrive from a PC.

8 The FAT partition and the `fat` command

Partition 1 of the SD card is ordinary FAT, readable and writable by any PC — this is how files travel to Fuzix, because nothing else can mount the Unix partition. Format it once in Windows (FAT or FAT32, **not exFAT**), then simply copy files onto it.

On the Fuzix side, the `fat` command reads it (long filenames and subdirectories included):

```
# fat info
/dev/hdb1: FAT16, 32695 clusters of 2048 bytes (62 MB)
# fat ls
    4523  My Program.bas
   <dir>  BASIC
# fat ls basic
# fat get "my program.bas" /root/prog.bas
# fat get demo.bin                    (lowercased name in current dir)
```

`fat` is read-only by design — the desktop does the writing. To move a file *out* of Fuzix, use the serial port, or write it with `doswrite` (the venerable Minix FAT12/16 tool, also included) if the partition is FAT16.

9 Moving files over the serial port

The FAT partition carries files *in*, but `fat` is read-only, so it cannot carry anything back *out*. The serial console can do both, in either direction, with nothing on the board but `uue` and `uud` — and it is the only route that needs no card swapping.

The idea is older than the machine: `uuencode` turns any file into printable text, and printable text survives a terminal. Both tools are in `/bin`:

```
# uue FILE            writes FILE.uue - the encoded text
# uud FILE.uue        decodes it back, recreating the original name
```

9.1 Sending a file to the board

On the PC, encode the file, then have the board read the text into a file and decode it:

```
# cat > prog.uue          (now paste or send the encoded text)
^D
# uud prog.uue
# ls -l prog.bas
```

The one thing that will bite you is flow control. The terminal input queue is 132 bytes. Sending encoded text as fast as the port will take it overruns that queue and loses characters silently — a 62-character line sent every 25 ms still loses roughly 60% of the text, and the damage only shows up as a corrupt file at the end. Two ways to avoid it:

- **Let the terminal pace it.** Any sender with a per-line delay *and* a wait for the echo will do. The delay alone is not enough.
- **Use the supplied script.** `devtools/uusend.py` in the source tree does exactly this: it types the text one line at a time and waits for each line to echo before sending the next, so at most one line is ever in flight. It then runs `uud` for you.

```
python uusend.py local.bas prog.bas --port=COM11
```

Note that `uusend.py` takes a **bare** remote filename — it drives one `ucp`-style session with `cd` and plain names, so `mv` the file afterwards if it belongs somewhere else.

9.2 Fetching a file from the board

The other direction is easier, because the board is the slow end:

```
# uue report.txt
# cat report.uue
```

Capture the session to a file in your terminal program, cut away anything before `begin` and after `end`, and decode it on the PC.

9.3 Programs to use

Linux (and macOS, and WSL):

- `uuencode` / `uudecode` come from the **sharutils** package (`apt install sharutils`). macOS has `uuencode` built in.
- For the terminal, **picocom** (simplest), **minicom** (its `Ctrl-A S` menu includes an `ascii-xfer` that paces lines), or **screen /dev/ttyUSB0 115200**.

- If sharutils is awkward, Python needs no install: `python3 -c "import binascii,sys; ..."` — or just use `devtools/uusend.py`, which does the encoding itself.

Windows:

- **Tera Term** is the pick — free, and its *File → Send file* has a per-line delay setting under *Setup → Serial port* (set a transmit delay of a few ms per character or 30–50 ms per line). This is the easiest reliable route without scripting.
- **PuTTY** works for capture (*Session → Logging*) but has no paced file send; pasting a large file into it will lose characters.
- **RealTerm** can send a file with a configurable delay, and is good at capturing binary.
- For the encode/decode step, Windows has no built-in `uuencode`. Use **WSL** (then it is the Linux instructions), **Python** for Windows, or **UUDeview**, which still builds and runs.

Whatever you use, the check at the end is the same: `sum` the file on both machines and compare. On the board, `sum FILE`.

10 The filesystem and included software

10.1 Layout

```
/bin          core utilities (always available)
/usr/bin      larger tools, BBC BASIC, fat, picoctl
/usr/games    the games collection
/usr/lib/bbc  BBC BASIC library directory (@lib$)
/usr/lib/cc   the C compiler passes, and its headers in include/
/usr/man      manual pages (man <name>)
/dev          devices - see below
/etc          rc (boot script), inittab, passwd, termcap
/tmp          scratch space
/root         root's home directory
```

Key devices:

Device	What it is
`/dev/tty1`	The console (HDMI + USB keyboard + serial mirror)
`/dev/tty2`	The GP0/GP1 serial port
`/dev/hda`	On-board NAND flash filesystem
`/dev/hdb1`–`hdb3`	SD card partitions (root is `hdb2`)
`/dev/rtc`	The DS3231 clock (`setdate` reads and sets it)
`/dev/sys`	Platform control (graphics, sound, ADVAL ioctls)

`/etc/rc` runs at boot: filesystem check, clock from the DS3231, keyboard layout. Edit it with any of the editors below.

There is no swap device to enable. The PSRAM is not a block device any more: the kernel takes an allocation the size of the process when it needs to swap one out, and programs take their heap from the same place. `free` reports both.

10.2 Applications

Editors: `mmedit` (MMBasic's own full-screen editor — see its chapter), `vi` (levee), `ue` (a WordStar-diamond micro-Emacs: Ctrl-W write, Ctrl-Q quit), `ed`, `fleamacs`.

Shell & core: Bourne `sh` (plus `fsh`), and the classic set — `ls ll cp mv ln rm mkdir rmdir cat more less head tail grep fgrep sed tr cut sort uniq wc find xargs diff diff3 cmp comm join split rev tar dd df du free ps kill killall uptime date cal banner echo sleep tee touch which who su passwd stty mount umount sync fsck mkfs fdisk chmod chown chgrp od hd factor seq units dc expr m4 make cron at mail write wall` (and more — see `ls /bin /usr/bin`).

Machine tools: `picoctl` (keyboard layout, reboot to the flasher), `picogpio/gpiotool`, `gfxtest` (display test card), `setdate` (DS3231), `flashrom`, `setboot`, `dosread/doswrite` (FAT12/16 floppy-era transfers), `fat`, `uue/uud` (serial file transfer — see that chapter).

Images: `saveimage` writes any part of the screen as a 24-bit BMP, `loadimage` puts one back:

```
# saveimage shot.bmp                    the whole screen
# saveimage part.bmp 160 120 320 240    x y w h
```

```
# loadimage shot.bmp                    at 0,0
# loadimage tiger.bmp 0 0 4             with dither mode 4
```

`loadimage` is MMBasic's BMP decoder, so it reads 1/4/8/16/24/32-bit files, `BI_BITFIELDS`, and RLE4/RLE8, and dithers only when asked. These are the programs `SAVE IMAGE` and `LOAD IMAGE` run, and they are just as useful from the shell.

Languages: `bbcbasic`, `cc` (the on-board C compiler — see its own chapter, with `cpp`, `bcrun` and `bcdump`), `fforth` (a complete ANS Forth), the `as09`/`ld09` assembler pair, and `dc`.

Games (`/usr/games`): the original Colossal Cave `advent`, the complete Scott Adams `adv01`–`adv14` and Mysterious Adventures `myst01`–`myst11` collections, Infocom Z-machine interpreters `z1`–`z8` and `l9x` (Level 9), `startrek`, `hamurabi`, `backgammon`, `invaders`, `2048`, `moo`, `ttt`, `fish`, `arithmetic`, `fortune`, `cowsay`, `wump`.

11 Appendix A: pin usage

GPIO	Function
GP0 / GP1	Serial port `/dev/tty2` (TX / RX)
GP8 / GP9	System console UART (USB-C CH340)
GP10/11/22	Audio I2S: BCLK / LRCLK / DATA (PCM5102 DAC)
GP12–GP19	HDMI (HSTX)
GP20 / GP21	I2C0: DS3231 RTC and QWIIC connector (SDA / SCL)
GP27	DS3231 32 kHz — board identification (PC3 only)
GP28/30/31/33	SD card, Pico Computer 3 (MISO/SCK/MOSI/CS, SPI1)
GP29/30/31/32	SD card, Pico Computer 2 (CS/SCK/MOSI/MISO, bit-banged)
GP32	DS3231 alarm interrupt (PC3; unused by Fuzix)
GP34–GP37	Joystick switches — `ADVAL(0)` bits 0–3
GP40	Analogue input (reserved; not read by ADVAL)
GP41–GP44	Analogue inputs — `ADVAL(1)`–`ADVAL(4)`
GP45/GP46	Analogue-capable, free
GP23/24/25/29	CYW43 wireless (PC3; unused by Fuzix)
GP47	PSRAM chip select

All spare header pins remain ordinary 3.3 V GPIO. Do not apply 5 V to any GPIO.

12 Appendix B: boot options

The kernel command line (held by the bootloader) accepts:

- `hda` / `hdb2` ... — root device override
- `kbd=us|uk|de|fr|es|be` — early keyboard layout override (the layout in `/etc/rc` then applies for the session)

The build, source and development notes live in the `pc3` branch of `github.com/UKTailwind/FUZIX` under `Kernel/platform/platform-rpipico/` — see `PC3-DEVNOTES.md` for the engineering history and `PC3-GFX-DESIGN.md` for the display design.

13 Appendix C: MMBasic coverage

This is what `mmbc` translates today. It is generated from the translator's own tables (`fcc/coverage.py` in the mmb2c repository), so it says what the program does rather than what anyone remembers it doing.

Coverage grows with each release, and it will never be complete. MMBasic is a large language whose statements reach deep into one particular firmware, and a translator that emits portable C cannot follow all of it. Anything not listed here is reported by name, with its line number, and the translation continues - so you find out at translate time, not at run time.

13.1 Statements

?	ARRAY	CALL	CASE
CAT	CHDIR	CLEAR	CLOSE
CONST	CONTINUE	COPY	DATA
DATE$	DIM	DO	ELSE
ELSEIF	END	ENDIF	ERASE
ERROR	EXIT	FILES	FOR
FUNCTION	GOSUB	GOTO	IF
INC	INPUT	KILL	LET
LINE	LOCAL	LONGSTRING	LOOP
CIRCLE	COLOUR	LOAD IMAGE	MATH
MKDIR	MODE	NEXT	ON
OPEN	OPTION	PAUSE	PIXEL
PRINT	RANDOMIZE	READ	RENAME
RESTORE	RETURN	RMDIR	SAVE IMAGE
SEEK	SELECT	SORT	STATIC
SUB	SYSTEM	TIME$	TIMER
WEND	WHILE		

Assignment needs no keyword (`LET` is accepted). Statement separators, line numbers and labels, `REM` and `'` comments all work as expected.

13.2 Functions

ABS	ACOS	ASC	ASIN
ATAN2	ATN	BIN$	BIN2STR$
BIT	BOUND	BYTE	CHOICE
CHR$	CINT	COS	CWD$
DATE$	DATETIME$	DAY$	DEG
DIR$	EOF	EPOCH	EXP
FIELD$	FIX	FORMAT$	HEX$
INPUT$	INSTR	INT	LCASE$
LCOMPARE	LEFT$	LEN	LGETBYTE
LGETSTR$	LINPUT	LINSTR	LLEN
LOC	LOF	LOG	LTRIM$
MATH	MAX	MID$	MIN
OCT$	PI	RAD	RGB

RIGHT$	RND	RTRIM$	SGN
SIN	SPACE$	SQR	STR$
STR2BIN	STRING$	TAB	TAN
TIME$	TIMER	TRIM$	UCASE$
VAL			

13.3 MATH() sub-functions

Scalar: ATAN3, COSH, LOG10, SINH, TANH

Whole-array (one number out of an array): MAX, MEAN, MEDIAN, MIN, SD, SUM

13.4 Types and structure

INTEGER (64-bit), FLOAT (double), STRING, and arrays of each, up to the dimensions MMBasic allows. DIM, LOCAL, STATIC, CONST, OPTION BASE, SUB and FUNCTION with by-reference arguments, and the usual control flow.

13.5 Not covered

Sound, GPIO, I2C, SPI, one-wire, interrupts, SETPIN, PIN and PORT - along with the editor, RUN, LIST, EDIT, LOAD, SAVE and the rest of the immediate-mode environment. The hardware statements are the subject of current work — the display arrived in v0.5 — while the immediate-mode ones will never apply, since a translated program is compiled and run rather than typed at a prompt. (mmedit provides the editing they existed for.)

Of the graphics, MODE, COLOUR, PIXEL, LINE, CIRCLE and RGB() are done; BOX, TRIANGLE, TEXT and the array form of PIXEL are not yet. In MODE 2 a program's PRINT output does not yet share the screen with the graphics as it does in MMBasic, and there is no OPTION CONSOLE to steer it; both are next.